Red Hat OpenShift Administration II: Configuring a Production Cluster

Course | Lab Environment

Lab: Enable Developer Self-service

Configure a project with restrictions that prevent its users and applications from consuming all capacity of a cluster.

Outcomes

- Configure project creation to use a custom project template.
- Limit resource usage for new projects.

As the `student` user on the `workstation` machine, use the `lab` command to prepare your environment for this exercise, and to ensure that all required resources are available:

```
[student@workstation ~]$ lab start selfservice-review
```

Instructions

The API URL of your OpenShift cluster is `https://api.lab.example.com:6443`, and the `oc` command is already installed on your `workstation` machine.

1. Log in to the OpenShift cluster as the `admin` user with `redhatocp` as the password.

1.1. Log in to the OpenShift cluster.

```
[student@workstation ~]$ oc login -u admin -p redhatocp \
https://api.lab.example.com:6443
Login successful.
...output omitted...
```

Hide Solution

2. Create the `template-test` namespace to design a project template, and configure the project template with the following properties:

- The user who requests the project has the default `admin` role binding.
- The workloads in the project cannot request a total of more than 2 GiB of memory, and they cannot use more than 4 GiB of memory.

Each workload in the project must have the following properties:

- Default memory request of 256 MiB
- Default memory limit of 512 MiB
- Minimum memory request of 128 MiB
- Maximum memory usage of 1 GiB

You can use the `oc create quota` command to create the resource quota without creating a YAML definition. A template for the limit range is available at `~/DO280/labs/selfservice-review/limitrange.yaml`.

> **NOTE**
>
> The next steps assume that you design the template in the `template-test` namespace. The lab scripts grade and clean the design namespace only if you create it with the `template-test` name.

2.1. Create the `template-test` namespace.

```
[student@workstation ~]$ oc create namespace template-test
namespace/template-test created
```

2.2. Create the `memory` quota in the `template-test` namespace. Set the `requests.memory` value to 2 GiB and the `limits.memory` value to 4 GiB.

```
[student@workstation ~]$ oc create quota memory -n template-test \
--hard requests.memory=2Gi,limits.memory=4Gi
resourcequota/memory created
```

2.3. Edit the `~/DO280/labs/selfservice-review/limitrange.yaml` file to set the limit range. Update the placeholder text to match the following content:

```
apiVersion: v1
kind: LimitRange
metadata:
  name: memory
  namespace: template-test
spec:
  limits:
  - min:
      memory: 128Mi
    defaultRequest:
      memory: 256Mi
    default:
      memory: 512Mi
    max:
      memory: 1Gi
    type: Container
```

2.4. Create the `memory` limit range by using the `~/DO280/labs/selfservice-review/limitrange.yaml` file.

```
[student@workstation ~]$ oc create \
-f ~/DO280/labs/selfservice-review/limitrange.yaml
limitrange/memory created
```

Hide Solution

3. Verify that the quota and limit range have the intended effect.

For example, create a deployment named `test-limits` that uses the `registry.lab.example.com:8443/redhattraining/hello-world-nginx:v1.0` image without resource specifications. The deployment has one replica.

Verify that the limit range adds requests and limits to the pods. Scale the deployment to 10 replicas. Examine the deployment and the quota to verify that they have the intended effect.

3.1. Use the `oc create` command to create the `test-limits` deployment. The deployment uses the `registry.lab.example.com:8443/redhattraining/hello-world-nginx:v1.0` container image. The deployment has one replica.

```
[student@workstation ~]$ oc create deployment -n template-test test-l
imits \
--image registry.lab.example.com:8443/redhattraining/hello-world-ngin
x:v1.0
deployment.apps/test-limits created
```

3.2. Run the following command to view the `resources` key of the container specification. Although you create the deployment without specifying resources, the limit range applies memory requests and limits.

```
[student@workstation ~]$ oc get pod -n template-test \
-o jsonpath='{.items[0].spec.containers[0].resources}'
{"limits":{"memory":"512Mi"},"requests":{"memory":"256Mi"}}
```

3.3. Scale the `test-limits` deployment to 10 replicas.

```
[student@workstation ~]$ oc scale deployment -n template-test test-li
mits \
--replicas 10
deployment.apps/test-limits scaled
```

3.4. Verify that the quota has an effect, and that the deployment scales to 8 replicas out of 10.

```
[student@workstation ~]$ oc get deployment -n template-test
NAME          READY   UP-TO-DATE   AVAILABLE   AGE
test-limits   8/10    8            8           8m41s
```

3.5. Describe the `memory` quota in the `template-test` namespace.

```
[student@workstation ~]$ oc describe resourcequota -n template-test m
emory
Name:            memory
Namespace:       template-test
Resource         Used  Hard
--------         ----  ----
limits.memory    4Gi   4Gi
requests.memory  2Gi   2Gi
```

The deployment uses the quota completely, and scales to only eight pods. Each pod requests 256 MiB of memory, and eight pods request 2 GiB of memory. Each pod has a 512 MiB memory limit, and eight pods have a 4 GiB memory limit.

Hide Solution

4. Create a project template definition with the properties that were specified in a previous step.

4.1. Create an initial project template by using the `oc adm create-bootstrap-project-template` command. Redirect the output to the `~/DO280/labs/selfservice-review/template.yaml` file.

```
[student@workstation ~]$ oc adm create-bootstrap-project-template \
-o yaml > ~/DO280/labs/selfservice-review/template.yaml
```

4.2. Review the `memory` limit range, and the `memory` quota resources created in a previous step. You use the output of this command in a following step.

> **NOTE**
>
> The solution for this step assumes that you designed your template in the `template-test` namespace.

```
[student@workstation ~]$ oc get limitrange,quota -n template-test \
-o yaml
apiVersion: v1
items:
- apiVersion: v1
  kind: LimitRange
  metadata:
    ...output omitted...
    name: memory
    namespace: template-test
    ...output omitted...
  spec:
    limits:
    - default:
        memory: 512Mi
      defaultRequest:
        memory: 256Mi
      max:
        memory: 1Gi
...output omitted...
```

4.3. Edit the `~/DO280/labs/selfservice-review/template.yaml` file and apply the following updates:

- Add the limit range and quota definitions from the previous step to the file, after the role binding definition.
- Update the `namespace` key value to `${PROJECT_NAME}` in the limit range and quota objects.

```
apiVersion: template.openshift.io/v1
kind: Template
...output omitted...
  subjects:
  - apiGroup: rbac.authorization.k8s.io
    kind: User
    name: ${PROJECT_ADMIN_USER}
- apiVersion: v1
  kind: LimitRange
  metadata:
    name: memory
    namespace: ${PROJECT_NAME}
  spec:
    limits:
    - default:
        memory: 512Mi
      defaultRequest:
        memory: 256Mi
      max:
        memory: 1Gi
      min:
        memory: 128Mi
      type: Container
- apiVersion: v1
  kind: ResourceQuota
  metadata:
    name: memory
    namespace: ${PROJECT_NAME}
  spec:
    hard:
      limits.memory: 4Gi
      requests.memory: 2Gi
parameters:
...output omitted...
```

A solution file is provided at `~/DO280/solutions/selfservice-review/template.yaml`.

Hide Solution

5. Create and configure the project template.

5.1. Create the project template by using the `~/DO280/labs/selfservice-review/template.yaml` file.

```
[student@workstation ~]$ oc create -n openshift-config \
-f ~/DO280/labs/selfservice-review/template.yaml
template.template.openshift.io/project-request created
```

5.2. Use the `oc edit` command to change the cluster project configuration.

```
[student@workstation ~]$ oc edit projects.config.openshift.io cluster
```

Edit the resource to match the following content:

```
apiVersion: config.openshift.io/v1
kind: Project
metadata:
...output omitted...
  name: cluster
spec:
  projectRequestTemplate:
    name: project-request
```

To edit the file, you use the default `vi` editor.

5.3. Run the `watch` command and wait until the API server pod restarts. Restarting the pod might take a few minutes.

```
[student@workstation ~]$ watch oc get pod -n openshift-apiserver
Every 2.0s: oc get pod -n openshift-apiserver                  ...

NAME                          READY   STATUS    RESTARTS   AGE
apiserver-57858f94b4-9tq9h    2/2     Running   0          2m29s
```

Press **Ctrl+C** to end the `watch` command after the API server pod displays `Running` in the `STATUS` column.

Hide Solution

6. Create a project named `template-validate` to verify that the template creates the quota and limit range objects.

6.1. Use the `oc new-project` command to create a `template-validate` project.

```
[student@workstation ~]$ oc new-project template-validate
Now using project "template-validate" on server "https://api.lab.exam
ple.com:6443".
...output omitted...
```

6.2. Describe the quotas in the project.

```
[student@workstation ~]$ oc describe quota
Name:            memory
Namespace:       template-validate
Resource         Used  Hard
--------         ----  ----
limits.memory    0     4Gi
requests.memory  0     2Gi
```

6.3. Describe the limit ranges in the project.

```
[student@workstation ~]$ oc describe limitrange
Name:       memory
Namespace:  template-validate
Type        Resource  Min    Max  Default Request  Default Limit  ...
----        --------  ---    ---  ---------------  -------------  ...
Container   memory    128Mi  1Gi  256Mi            512Mi          ...
```

> **NOTE**
>
> Optionally, you can execute the commands in Step 3 again to verify that the quota and limit range work as expected. Ensure that you use the `template-validate` project.

Hide Solution

Evaluation

As the `student` user on the `workstation` machine, use the `lab` command to grade your work. Correct any reported failures and rerun the command until successful.

```
[student@workstation ~]$ lab grade selfservice-review
```

Finish

As the `student` user on the `workstation` machine, use the `lab` command to complete this exercise. This step is important to ensure that resources from previous exercises do not impact upcoming exercises.

```
[student@workstation ~]$ lab finish selfservice-review
```

Revision: do280-4.22-ed1-20260827-78c07d2



Privacy Policy (https://www.redhat.com/en/about/privacy-policy?extIdCarryOver=true&sc_cid=701f2000001D8QoAAK)

Red Hat Training Policies (https://www.redhat.com/en/about/red-hat-training-policies)

Terms of Use (https://www.redhat.com/en/about/terms-use)

All policies and guidelines (https://www.redhat.com/en/about/all-policies-guidelines)

Release Notes (https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

Cookie Preferences and Opt-Out Rights